

Prontuario FI2 — da tenere aperto all'esame

Cos'è. Il punto d'ingresso del kit d'esame (esame_FI2/). Non spiega: *risponde*, e quando la risposta è un pezzo di codice rimanda al **file dove Lorenzo l'ha già scritto**.

Come si usa in prova. 1. **Errore rosso** in Eclipse o eccezione in console → Ctrl+F sul testo del messaggio (§1). 2. **Devi fare una cosa** (leggere un file, ordinare, arrotondare) → *Indice per bisogno*. 3. ▲ segna le righe dove Lorenzo ha già sbagliato almeno una volta: lì rallenta.

Com'è ordinato. Per **parte del compito**, come i package dello startkit d'esame (model, persistence, controller, ui), non per modulo del corso: in prova sai in quale classe sei, non da quale slide viene la cosa. [riorganizzato il 2026-09-30]

Come cresce. A ogni fine sessione pratica (/chiudi, passo 8a), solo con ciò che è stato **incontrato davvero**; ogni frammento di codice è commentato riga per riga. Le sezioni vuote restano visibili: si riempiono coi LAB e con le prove.

Fonti. [02x sl. N] = pagina del PDF di slide indicato. [oltre la fonte] = non è nelle slide, verificato in jshell (JDK 21) e, per i messaggi di Eclipse, con il suo compilatore (ecj 3.38), il 2026-09-26.

Indice per bisogno

Devo...	Vai a	Fatto in
capire un errore «Type mismatch» / «lossy conversion»	1.1	02x
capire un'eccezione in console (/ by zero, NumberFormatException)	1.2	02x
capire perché un risultato è sbagliato senza nessun errore	1.3	02x
sistemare Eclipse che non compila al primo avvio	2	setup 26/09
sapere quanto è grande / fin dove arriva un tipo	3.1	02x
sapere se un assegnamento fra numeri compila	3.2	02x
capire overflow, Infinity, NaN; confrontare due double	3.3	02x

Devo...	Vai a	Fatto in
arrotondare a intero	3.4	02x
trasformare una stringa in numero, o un numero in stringa	3.5	02x
fare conti su caratteri ('7' → 7, lettera successiva)	3.6	02x
scrivere un main e leggere gli argomenti	3.7	02x
importare lo startkit, rinominare il progetto	2	LAB02
lanciare test fatti di assert (-ea); capire le X rosse iniziali	2 · 1.1	LAB02
scrivere una classe-valore immutabile: costruttori, getter, equals, toString	3.8	LAB02
usare mcd / scrivere mcm	3.8	LAB02
operazioni che restituiscono un oggetto nuovo (sum, sub, mul, div, reciprocal)	3.9	LAB03
capire l'mcm e portare due frazioni allo stesso denominatore	3.9	LAB03
scrivere un compareTo che restituisce 0 / 1 / -1	3.9 · 1.3	LAB03
dividere due int e ottenere un double	2 · 1.2	LAB03
capire UnsupportedOperationException al lancio	2 · 3.10 · 1.1	LAB04a
spostare classi in package, sistemare gli import	3.10	LAB04a
scrivere una libreria di funzioni static; capire quando un metodo è static	3.10	LAB04a
scorrere un array di oggetti e accumulare un risultato (somma, prodotto)	1.1 · 3.10	LAB04a
capire «non-static method ... static context»	3.11	LAB04b
scorrere un array		
riempito a metà (celle null in fondo): condizione del ciclo		

Devo...	Vai a	Fatto in
contare gli elementi di un array a metà (size)	3.11	LAB04b
creare e restituire un array nuovo (somma cella per cella di due array)	3.11	LAB04b
stampare un array come [a, b, c] senza virgola finale	3.11	LAB04b
metodi static e d'istanza nella stessa classe; tre sum con lo stesso nome (overloading)	3.11	LAB04b
nascondere un array dentro una classe (ADT con campi private, costanti static final)	3.12	LAB04c
scrivere più costruttori senza duplicare codice (this(...))	3.12	LAB04c
costruire un oggetto da un array che mi passano, copiandolo (non tenendo il riferimento)	3.12	LAB04c
put che aggiunge in coda e raddoppia l'array quando è pieno	3.12	LAB04c
remove(i): spostare le celle indietro di uno, azzerare l'ultima, size--	3.12	LAB04c
lanciare un'eccezione (throw new ...) per un indice o un argomento non valido	3.12	LAB04c
toString di una collezione con StringBuilder ([a, b, c], [])	3.12	LAB04c
operazione fra due collezioni che restituisce una collezione nuova (sum, mul)	3.12	LAB04c
formattare un importo in euro (3,50 €)	3.13	LAB05
formattare un orario (08:30) o una durata (6:30)	3.13	LAB05

Devo...	Vai a	Fatto in
durata fra due orari, in minuti totali	3.13	LAB05
applicare una regola «sottrai, poi controlla il minimo» (franchigia + durata minima)	3.13	LAB05
capire cosa va nel costruttore e cosa nei parametri di un metodo	3.13	LAB05
scegliere un elemento di un array in base al giorno della settimana	3.13	LAB05 (parte 2)
scorrere i giorni fra due date (un giorno alla volta, con un <code>LocalDate</code> che avanza)	3.13	LAB05 (parte 2)
calcolare una sosta su più giorni con tariffe diverse per giorno	3.13	LAB05 (parte 2)
aggiungere giorni/minuti a una data (<code>plusDays</code> , <code>plusMinutes</code>) senza che il risultato si perda	3.13	LAB05 (parte 2)

1 — Errori → causa → rimedio

Una sola tabella per tutto il corso: cerca il testo del messaggio con Ctrl+F.

1.1 Errori di compilazione

Il testo di Eclipse è quello che vedi all'esame; quello di `javac` è quello di `jshell` e del terminale. **Stessa causa, parole diverse.**

Eclipse	<code>javac</code> / <code>jshell</code>	Causa	Rimedio
Type mismatch: cannot convert from double to float	incompatible types: possible lossy conversion from double to float	letterale senza F (3.54 è double), o espressione con un double dentro	3.54F, oppure cast (float) espr — △ vedi 3.2

Eclipse	javac / jshell	Causa	Rimedio
Type mismatch: cannot convert from long to int	... possible lossy conversion from long to int	letterale con L; oppure Math.round(double), che restituisce long	(int) Math.round(x)
Type mismatch: cannot convert from double to int	... lossy conversion from double to int	letterale con punto (3.0); oppure Math rint, sqrt, pow, che restituiscono double	(int) espr — ma tronca: vedi 3.4
Type mismatch: cannot convert from int to short / to byte	... lossy conversion from int to short	aritmetica su byte/short produce int (c + 1); oppure si assegna una <i>variabile</i> int	(short)(c + 1) — parentesi attorno all'intera espressione
Type mismatch: cannot convert from int to char	... lossy conversion from int to char	ch + 1 è un int	(char)(ch + 1), oppure ch++ (compila: ha il cast incorporato)
Type mismatch: cannot convert from String to int	String cannot be converted to int	gli argomenti del main sono stringhe, non numeri	Integer.parseInt(s) / Double.parseDouble(s) — vedi 3.5
The literal 3000000000 of type int is out of range	integer number too large	un letterale senza L è int, e non arriva a 3·10 ⁹	3000000000L
Frazione cannot be resolved to a type	cannot find symbol ... symbol: class Frazione	la classe non esiste ancora: sono le X rosse normali dello startkit	scriverla col nome esatto usato nei test, nel package dei test
MyMath cannot be resolved (classe di un altro package)	cannot find symbol ... symbol: variable MyMath	la classe esiste, ma sta in un altro package e manca l'import	import util.MyMath; sotto la riga package [08 sl. 18; LAB04 sl. 18]
Cannot make a static reference to the non-static method sum(Frazione) from the type Frazione	non-static method sum(Frazione) cannot be referenced from a static context	metodo d'istanza chiamato sulla classe (Frazione.sum(f)): manca l'oggetto che fa da this	chiamarlo su un oggetto (a.sum(f)), oppure è una funzione da libreria → 3.10

Eclipse	javac / jshell	Causa	Rimedio
This method must return a result of type int	missing return statement	metodo dello startkit con solo // da fare, o un ramo senza return	un return su ogni strada del metodo
The method sum(Frazione) in the type Frazione is not applicable for the arguments (Frazione[], Frazione[])	method sum in class Frazione cannot be applied to given types	esiste un sum con quel nome ma con altri argomenti: il test chiama un overload non ancora scritto	scrivere il metodo con la firma del test (qui static Frazione[] sum(Frazione[], Frazione[])) → 3.11
The method convertToString(Frazione[]) is undefined for the type Frazione	cannot find symbol ... symbol: method convertToString(Frazione[])	nessun metodo con quel nome nella classe	scriverlo, nella classe che il test usa come prefisso
⚠ The method Size(Frazione[]) is undefined for the type Frazione (testo Eclipse per analogia con la riga sopra, non rilanciato)	cannot find symbol ... symbol: method Size(Frazione[]) · location: class Frazione (verificato con javac)	maiuscole: Java distingue Size da size; il metodo esiste, scritto minuscolo	Frazione.size(...)
⚠ The constructor Ticket(double, LocalDateTime, LocalDateTime) is undefined	incompatible types: double cannot be converted to LocalDateTime	i parametri del costruttore sono in ordine diverso da quello che il test (o il chiamante) passa. Eclipse nomina i tipi del tuo costruttore	l'ordine lo detta il test: leggi new Ticket(...) in TicketTest e allinea la firma → 3.13
The constructor Parcometro() is undefined	constructor Parcometro in class Parcometro cannot be applied to given types	si chiama new Parcometro() ma la classe ha solo il costruttore con la Tariffa	passare l'argomento richiesto: new Parcometro(tariffa)

1.2 Eccezioni a run-time

Compila tutto, esplode eseguendo — in console, in rosso.

Eccezione	Quando	Rimedio
ArithmeticException: / by zero	divisione o % fra interi per zero. Fra reali non succede: dà Infinity/NaN [02x sl. 21-22]	controllare il divisore prima
AssertionError ... at FrazioneTest.st.main(FrazioneTest.java:28) △ ArithmeticException: / by zero ... at MyMath.mcd	con -ea, un assert del test è falso: è il failure di un test fatto di assert mcd(0, n): Euclide scambia e fa n % 0 [LAB02 sl. 9]	apri la riga indicata, leggi cosa si aspetta, prova quel caso a mano nel chiamante, gestire il numeratore 0 prima di chiamare mcd; non modificare mcd
NullPointerException: Cannot read field "den" because "f" is null ... at frazione.Frazione.sum ... at frazlib.FrazLib.sum △	una cella dell'array è null: new Frazione[4] crea 4 caselle vuote , non 4 frazioni [07 sl. 17]. È un error , non un failure	riempire ogni cella (fs[i] = new Frazione(...)) prima di passare l'array. Leggi la traccia dal basso: chi ha passato il null
NullPointerException: Cannot invoke "frazione.Frazione.toString()" because "fs[i]" is null △ ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3 ... at FractionCollection.remove △ ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0 ... at FractionCollection.put	un ciclo su un array riempito a metà è arrivato alla prima cella vuota: la condizione controlla solo length ciclo di spostamento i < size su array pieno : all'ultimo giro legge innerContainer[i+1] = [size], oltre la fine capacità 0: «il doppio» di 0 è 0, e si scrive in [size]	i < fs.length && fs[i] != null → 3.11 i < size - 1 → 3.12
△ ArrayIndexOutOfBoundsException: Index -1 out of bounds for length 10 ... at FractionCollection.toString △ la collezione «perde» l'elemento aggiunto dopo che l'array era pieno, senza errori	l'ultimo elemento scritto fuori dal ciclo con innerContainer[size-1]: con size 0 è l'indice -1 nel ramo del raddoppio il nuovo array resta in una variabile locale: manca innerContainer = fs;	if (innerContainer.length == 0) crea un array da DEFAULT_PHYSICAL_SIZE prima del raddoppio → 3.12 tutti gli elementi dentro il ciclo, virgola prima di ogni elemento tranne il primo → 3.12 assegnarlo al campo → 3.12

Eccezione	Quando	Rimedio
△ <code>ArrayIndexOutOfBoundsException</code>: Index 2 out of bounds for length 2 / Index -1 out of bounds for length 0 <code>NumberFormatException</code>: For input string: "..."	<code>fs[fs.length]</code> (l'ultimo indice valido è <code>length-1</code>), oppure <code>fs[fs.length-1]</code> su un array vuoto <code>parseInt/parseDouble</code> su stringa non valida — le virgolette nel messaggio mostrano la stringa esatta: guardale per vedere spazi e virgole al lancio, prima di eseguire qualsiasi riga: Eclipse ha compilato per una Java più recente del JRE che esegue.	non stampare «l'ultimo» a parte: separatore <i>prima</i> di ogni elemento tranne il primo → 3.11 controllare la stringa prima di convertirla [02x sl. 35] — vedi 3.5
△ <code>LinkageError</code> durante il caricamento della classe principale <code>X.java.lang.UnsupportedClassVersionError</code>: X has been compiled by a more recent version of the Java Runtime (class file version 69.0), this version of the Java Runtime only recognizes class file versions up to 65.0	Versione class file = Java + 44: 65 = Java 21, 69 = Java 25	compliance del compilatore = versione del JRE → §2

1.3 Errori silenziosi

Compila, gira, e il risultato è sbagliato. Nessun messaggio.

Sintomo	Causa probabile	Vai a
numero enorme negativo, o segno invertito	overflow	3.3
<code>x == y</code> falso fra due <code>double</code> che «dovrebbero» essere uguali (<code>int</code>) 3.9 dà 3	errore numerico	3.3
<code>Math rint(2.5)</code> dà 2.0 test fatti di <code>assert</code> : console vuota anche col codice sbagliato	il cast tronca , non arrotonda <code>rint</code> arrotonda al pari manca <code>-ea</code> : senza, gli <code>assert</code> non vengono eseguiti	3.4 3.4 2

Sintomo	Causa probabile	Vai a
△ oggetto con campi a 0 (new Frazione(0, -5) → 0/0)	catena if / else if senza else: per un input nessun ramo assegna, e i campi restano al default (0). Java non lo segnala, perché i campi sono già inizializzati	3.8
△ test verdi ma metodo sbagliato (mcm(4,6) = 22)	nessun test lo copre: verde vuol dire solo che i casi del test passano	provare a mano 2-3 casi, compreso lo 0
△ getDouble() dà 0.0 per ogni frazione fra -1 e 1	double v = num / den;; int / int è divisione intera , e la conversione a double arriva dopo, sul risultato già troncato	(double) num / den — 3.9
△ somma via mcm che «perde» un addendo (1/4 + 1/8 = 1/8)	fattore scritto al contrario: den / mcm invece di mcm / den; fra int fa 0	3.9
△ test verde per caso (sub, compareTo)	due errori che si compensano sul caso del test, o un caso che non distingue (3/12 e 1/4 → entrambe 0.0)	aggiungere un caso scelto da te: per compareTo sia 1 sia -1
△ somma di un array pieno che «perde» l'ultimo elemento ({1/2, 1/3} → 1/2)	ciclo i < fs.length - 1: il -1 salta l'ultima cella, non evita i null	i < fs.length (+ && fs[i] != null se l'array è a metà) → 3.11
△ 0/36 invece del 0/6 atteso dal test	somma col prodotto in croce (den·den) invece che con l'mcm; e minTerm lascia lo zero com'è	sommare via mcm (sumWithMcm / la sum del docente) → 3.9
△ test verdi ma costo sbagliato dopo una franchigia (H1f, 90 minuti → 0,25 € invece di 0,50 €)	durata minima confrontata con la durata totale invece che con quella dopo la franchigia: i test dello startkit non hanno un caso che li distingue	confrontare la variabile calcolata dopo la sottrazione; provare a mano un caso in cui la franchigia porta sotto il minimo → 3.13

2 — Eclipse e procedura d'esame

Problema	Rimedio	Incontrato
Al primo avvio: il JRE selezionato non supporta il <i>compliance level 25</i>	<i>Window → Preferences → Java → Compiler → Compiler compliance level = 21</i> [S01 p. 27]	Eclipse 2026-09 + JDK 21, 26/09
⚠ Startkit importato: al Run UnsupportedClassVersionError (class file 69 vs 65)	lo startkit non fissa la compliance e prende quella di default (25), ma il JRE è 21. Workspace: <i>Window → Preferences → Java → Compiler → 21 → Apply and Close → rebuild Yes</i> (o <i>Project → Clean...</i>). Solo il progetto: tasto destro → <i>Properties → Java Compiler → Enable project specific settings → 21</i>	LAB03, 30/09

Import dello startkit: **Next** non si attiva | è la seconda pagina del wizard: il pulsante è **Finish**. «Select root directory» → Browse → se il riquadro *Projects* è vuoto, **Refresh** → **spunta** il progetto → Finish [LAB02 sl. 16-18] | LAB02, 30/09 |

Procedura completa (si perdono punti se manca il rename) | scompatta lo zip → **rinomina la cartella** → *File → Import → General → Existing Projects into Workspace → Refactor → Rename* del **progetto**, con l'aggiornamento dei riferimenti spuntato [LAB02 sl. 15-22] | LAB02, 30/09 |

Refactor → Rename rifiuta il nome | il progetto sta dentro il workspace ed esiste già una **cartella con quel nome** (quella rinominata a mano), dove Eclipse vorrebbe spostarlo → dare al progetto un nome **diverso** da quello della cartella; Eclipse rinomina anche la cartella | LAB02, 30/09 |

Il nome che conta | quello scritto in `.project (<name>...</name>)`, che Eclipse aggiorna col Refactor; rinominare la cartella a mano non lo cambia | LAB02, 30/09 |

Test fatti di assert in un main (non JUnit) | tasto destro sulla classe di test → *Run As → Run Configurations...* → *Arguments → VM arguments = -ea* → Run; poi basta ►. Console vuota = tutti passati, **solo con -ea** [LAB02 sl. 12, 24-25] | LAB02, 30/09 |

X rosse appena importato | normali: mancano le classi e i metodi da scrivere [LAB02 sl. 23] | LAB02, 30/09 |

Ripartire dal LAB precedente | tasto destro sul progetto → *Copy → Paste* → nuovo nome. La configurazione di Run con `-ea` va **ricreata** sul progetto nuovo: controlla con un assert `false`; provvisorio che la console dia `AssertionError` | LAB04a, 01/10 |
Classi in package (sl. 19) | tasto destro su `src` → *New → Package* (`util`, `frazione`, `frazlib`); poi trascina la classe sul package, o *Refactor → Move*: Eclipse riscrive la riga `package` e gli import di chi la usa. Alla domanda sui *potential matches* → accetta | LAB04a, 01/10 |

Import di uno zip che fallisce | scompatta e usa *Existing Projects → Select root directory + Copy projects into workspace* | LAB04a, 01/10 |

Collaudo | **metodo per metodo**: commenta i blocchi di test non ancora pertinenti e

scommentali man mano, per evitare «errori in cascata» [LAB02 sl. 23]. Un assert fallito ferma il main: quelli dopo non sono ancora stati controllati | LAB02, 30/09 |

3 — Model

Dalla 02x: fonti materiali/slide/02x-x1-Esercitazione Tipi base.pdf (42 pagine) e 02z-Addendum-Main in Java21.pdf (7 pagine); il perché sta in lezioni/guida_lab_02x_tipi_base.md.

3.1 I tipi primitivi

Tipo	Byte	Range	Letterale	Note
byte	1	−128 ... 127	—	[02x sl. 11]
short	2	−32 768 ... 32 767	—	[02x sl. 11]
char	2	0 ... 65 535 (senza segno)	'A' apici singoli	Unicode, UTF-16 [02x sl. 23]
int	4	$\approx \pm 2,1 \cdot 10^9$ (Integer.MAX_VALUE = 2 147 483 647)	42	default dei letterali interi [02x sl. 11]
long	8	$\approx \pm 9,2 \cdot 10^{18}$	42L	«le costanti long terminano con la lettera L» [02x sl. 11]
float	4	fino a $\approx 3,4 \cdot 10^{38}$	3.54F	6–7 cifre significative [02x sl. 14]
double	8	fino a $\approx 1,8 \cdot 10^{308}$	3.54	14–15 cifre significative. Default dei letterali con punto [02x sl. 14]
boolean	—	true / false	—	tipo primitivo [02x sl. 27]

Il letterale ha un tipo, e lo decide la **sintassi**, non il valore: 42 → int, 42L → long, 3.54 → double, 3.54F → float, 'A' → char, "A" → String.

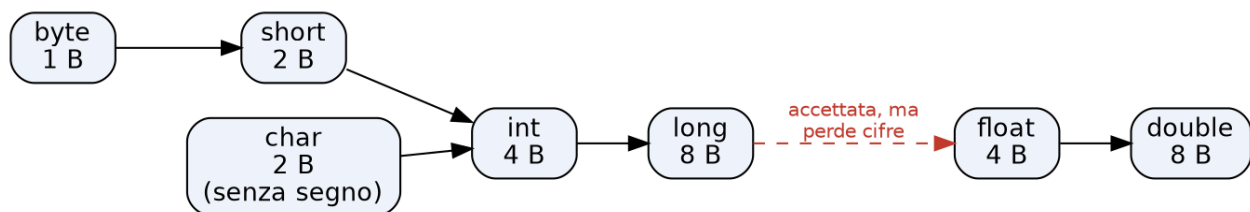


Figura 1: Conversioni allarganti: ogni freccia (e ogni cammino di frecce) è accettata senza cast

3.2 Conversioni fra numerici: compila o no?

Si legge nel verso delle frecce: se da X a Y c'è un cammino, $Y \text{ var} = \langle X \rangle$ compila senza cast. Contro il verso serve il cast. La freccia tratteggiata è l'unico punto dove «compila» e «non perde niente» smettono di coincidere (riga 5 sotto). Intercetta l'errore del verso invertito — $\triangle$ trasversale n. 5.

Il procedimento in 2 passi — sempre in quest'ordine: 1. **Tipo del lato destro:** letterale $\rightarrow$ tabella B; espressione $\rightarrow$ il tipo **più largo** fra gli operandi, e **mai meno di int** (short + int è int, ma anche byte + byte). 2. **Cammino nel diagramma** dal tipo di destra a quello di sinistra? Sì $\rightarrow$ compila. No $\rightarrow$ cast.

«In Java, C#, Scala sono ammessi solo gli assegnamenti che non causano perdita di informazione.» double x = 3.54F; lecita, float f = 3.54; illecita. [02x sl. 15]

Eccezioni al procedimento, tutte provate nel drill:

#	Caso	Esito	Perché
1	short s = 300; char c = 65; byte b = 10;	✓ compila	costante nota al compilatore e ci sta nel tipo
2	byte b = 300;	✗	costante, ma non ci sta
3	short s = 40000; int q = 10; byte b = q;	✗	q è una variabile: il compilatore non ne segue il valore
4	byte b = 127; b++;	✓ compila $\rightarrow$ b = -128	++ ha il cast incorporato: avvolge senza avvisare [02x sl. 13]
5	float h = 123456789L;	✓ compila $\rightarrow$ stampa 1.2345679E8	long $\rightarrow$ float è ammessa (conta il <i>range</i>), ma float tiene solo 7 cifre

Il cast — (tipo) espressione: è la dichiarazione di **Design Intent**, cioè la perdita è

una scelta tua e ne rispondi tu. ⚠ Usa questo termine all'orale (trasversale n. 4).

```
float f = (float) 3.54;           // double → float: perdita dichiarata, compila
int t = (int) 3.9;               // t = 3: il cast TRONCA, non arrotonda
int u = (int) -3.9;              // u = -3: tronca verso lo zero, non verso il basso
short d = (short)(c + 1);        // parentesi esterne obbligatorie: c + 1 è un int
// short d = (short) c + 1;      // SBAGLIATO: il cast lega solo a c, la somma torna
```

3.3 Aritmetica: le trappole che compilano

Overflow — Java protegge dalle *conversioni* che perdono dati, **non** dall'overflow aritmetico: il valore avvolge in silenzio [02x sl. 12-13].

Espressione	Valore
Integer.MAX_VALUE + 1	-2147483648
byte b = 125; b++; ++b; ++b;	-128 (da 127 passa a -128)
short e = c + 1; con c short	X non compila — qui c'è una conversione int → short, e il compilatore la ferma [02x sl. 12]

Reali e IEEE-754 — «Lo standard IEEE-754 incorpora le nozioni di infinito e Not-A-Number (NaN)» [02x sl. 21]; «I tipi interi non seguono lo standard IEEE-754» [02x sl. 22].

Espressione	Valore
5.0 / 0	Infinity
-5.0 / 0	-Infinity
0.0 / 0.0 · Math.sqrt(-1)	NaN
0.0 / -5	-0.0 — «Java mantiene il segno nel risultato, MA ciò non ne altera la semantica» [02x sl. 20]: -0.0 == 0.0 è true
5 / 0.5 % 0	ArithmeticException: / by zero
Double.NaN == Double.NaN	false — per testare usa Double.isNaN(x) [oltre la fonte]

Confrontare due double: mai con ==. Il collaudo delle equazioni avverte «soluzioni coincidenti - occhio agli errori numerici» [02x sl. 39]: il discriminante che «dovrebbe» valere 0 può valere 1e-16.

```
// 0.1 + 0.2 == 0.3 → false (0.1 + 0.2 vale 0.30000000000000004)
final double EPS = 1e-9;           // tolleranza: quanto vicino basta per dire '
if (Math.abs(delta) < EPS) {        // delta "è zero" se sta entro la tolleranza
    // radici coincidenti
}
```

3.4 Arrotondare

Funzione	Restituisce	2.5	-2.5	2.7	Per assegnarla a int
(int) x	int	2	-2	2	— (tronca verso zero)
Math.round(x)	long (da double)	3	-2	3	(int) Math.round(x)
Math rint(x)	double	2.0	-2.0	3.0	(int) Math.rint(x)

- [02x sl. 19]: «rint arrotonda un valore reale all'intero più vicino (risultato reale); round arrotonda un valore reale all'intero più vicino (risultato intero)». Sui valori **esattamente a metà** divergono: rint va al **pari** (0.5→0, 1.5→2, 2.5→2, 3.5→4), round va verso $+\infty$ (2.5→3, -2.5→-2). [oltre la fonte]
- Altre di Math [02x sl. 10, 19]: sqrt, pow(x, y) (→ double: pow(2, 10) = 1024.0), sin, hypot(x, y) = $\sqrt{x^2+y^2}$ «senza errori di overflow/underflow intermedi», log1p(p) = $\ln(1+p)$, la costante Math.PI.

3.5 Stringa ↔ numero

Stringa → numero [02x sl. 34]: Integer.parseInt, Double.parseDouble. «MA questi metodi pretendono stringhe “giuste”! altrimenti... BOOM!» [02x sl. 34]

Input	Integer.parseInt	Double.parseDouble
"42"	42	42.0
"3.54"	X NumberFormatException	3.54
"3,54" (virgola)	X	X — il separatore è il punto
"aa"	X	X

In Java «per evitare tale errore si può solo controllare la stringa prima di convertirla» [02x sl. 35].

```
double a = Double.parseDouble(args[0]); // args[0] è una String: va convertita
// con args[0] = "aa" qui il programma si ferma con NumberFormatException
```

Numero → stringa

```
String s1 = String.valueOf(3.0); // "3.0" — funzione statica [02x sl. 23]
String s2 = "" + 3.0;           // "3.0" — la concatenazione converte da solo
String s3 = String.valueOf('A'); // "A" — anche da char
```

Concatenazione con +: «l'operatore + concatena stringhe e nel farlo converte anche in stringa ciò che stringa non è» [02x sl. 7]. Si valuta **da sinistra**, e finché non compare una stringa è somma:

Espressione	Valore
"a" + 1 + 2	"a12"
1 + 2 + "a"	"3a"
"x" + 'a' + 'b'	"xab"
'a' + 'b' + "x"	"195x" — due char sommati sono un int (97 + 98)

3.6 Caratteri

char è un numero da 16 bit con segno di carattere: si somma, si confronta, si converte.

Espressione	Valore	Uso
(int) 'A' · int m = 'A';	65	codice del carattere [02x sl. 23]
char c = 65;	'A'	costante che ci sta → nessun cast (3.2, riga 1)
'7' - '0'	7	cifra-carattere → valore —
'a' + 1	98 (int!)	
(char)('a' + 1)	'b'	lettera successiva
ch++	'b'	compila senza cast (3.2, riga 4)

Character, classe-libreria — «non si può chiedere a un valore primitivo di “fare qualcosa”... bisogna chiamare una qualche funzione di una qualche libreria» [02x sl. 27]:

Chiamata	Valore	Nota
Character.toUpperCase('a') / toLowerCase	'A'	[02x sl. 28]
Character.isWhitespace(' ')	true	spazio, tab, a capo... [02x sl. 28]
Character.digit('8', 10) · digit('8', 7)	8 · -1	[02x sl. 29]
Character.digit('B', 16) · digit('B', 10)	11 · -1	-1 = «segnale di errore»: va controllato [02x sl. 29]

Integer [02x sl. 30]: signum(-42) → -1; rotateLeft(6, 1) → 12; rotateRight(6, 2) → -2147483647 (i bit usciti a destra rientrano a sinistra: è una *rotazione*, non uno shift).

Unicode, UTF, e i tre numeri di un carattere [02x sl. 23-26]. «In Java, i caratteri sono Unicode (16 bit, UTF-16)»; i byte UTF si ottengono «partendo da una stringa, NON da un carattere», con getBytes (...). ⚠ char e byte non stanno sullo stesso piano (occorrenza del 16/09).

Per U+1F608 (faccina, fuori dal piano base)	Valore	Risponde a
<code>s.length()</code>	2	quanti char (unità UTF-16: coppia surrogata)
<code>s.getBytes("UTF-8").length</code>	4	quanto occupa in UTF-8
<code>s.codePointCount(0, s.length())</code>	1	quanti caratteri veri
<code>"À".getBytes("UTF-8").length</code>	2	—

BOM di UTF-16. `"A".getBytes("UTF-16")` → [-2, -1, 0, 65] = FE FF 00 41. I due byte in testa sono il BOM U+FEFF. Messi nell'ordine FE FF, indicano **big-endian**: è la sl. 25 a essere corretta («Everything in Java is stored in big-endian order»), mentre la didascalia della sl. 24 («FE, FF è il marcatore little endian») è **imprecisa**. Verificato in jshell il 2026-09-26. UTF-8 → [65], UTF-32 → [0, 0, 0, 65].

3.7 main e argomenti

```
public class Prog {                                     // in Java il main sta sempre
    public static void main(String[] args) {           // static: esiste per tutto il
        if (args.length == 0)                          // length è una PROPRIETÀ dell'
            System.out.println("Nessun argomento");
        for (int i = 0; i < args.length; i++)           // da 0: args[0] è il primo argo
            System.out.println("argomento " + i + ": " + args[i]); // + converte
    }
}
```

- static «perché deve esistere dall'inizio alla fine del programma»; public «perché deve essere visibile dall'esterno» [02x sl. 2]. In C `argv[0]` è il nome del programma, in Java no [02x sl. 3].
- `java Prog alfa "beta gamma"` → 2 argomenti: le virgolette tengono insieme [02x sl. 9].
- Gli argomenti sono **sempre String**: per i numeri serve `parseDouble` — vedi 3.5 [02x sl. 37].
- Java 21 (preview) accetta anche `void main()` senza classe né argomenti, compilando con `javac --enable-preview --source 21`. Ordine di ricerca: statico con argomenti → statico senza → d'istanza con argomenti → d'istanza senza [02z, sl. 7]. All'esame usa la forma classica.

3.8 Classe-valore immutabile (ADT)

Lo schema della Frazione di LAB02, **nella forma del docente**, verificato coi test del LAB. Il perché di ogni scelta: `esame_FI2/svolti/LAB02_Frazione/confronto_LAB02.md`.

```
public class Frazione {                                // ADT "valore": una volta
    private int num, den;                             // private: da fuori si

    public Frazione(int num, int den) {                // costruttore PRIMARIO:
        boolean negativo = num * den < 0;             // segni opposti ⇒ prodot
        this.num = negativo ? -Math.abs(num) : Math.abs(num); // il segno sta SOLO
        this.den = Math.abs(den);                     // fuori da ogni ramo: se
```

```

    }

    public Frazione(int num) {                                // costruttore AUSILIARIO
        this(num, 1);                                         // delega al primario; de
    }

    public int getNum() { return num; }                       // accessor: niente set*,
    public int getDen() { return den; }

    public boolean equals(Frazione f) {                       // "uguale" = equivalente
        return f.getNum() * getDen() == f.getDen() * getNum(); // il confronto È gi
    }

    public Frazione minTerm() {                                // restituisce una frazi
        if (getNum() == 0) return new Frazione(getNum(), getDen()); // mcd(0, n) →
        int mcd = MyMath.mcd(Math.abs(getNum()), getDen()); // mcd vuole naturali
        return new Frazione(getNum() / mcd, getDen() / mcd);
    }

    @Override                                                 // ridefinisce il toString
    public String toString() {
        return getDen() == 1 ? "" + getNum() : getNum() + "/" + getDen(); // "4" se
    }
}

```

- Costruttori: primario + ausiliario che delega con this(...) [LAB02 sl. 5-6; 04b sl. 79-82]. Operatore cond ? a : b [06 sl. 96]. toString e @Override [06 sl. 24-26].
- toString: la slide chiede Num/Den [LAB02 sl. 10]; il docente stampa 4 per 4/1 (qui in una riga, stessa logica). **All'esame decide il testo del compito.**
- ⚠ **Ogni campo assegnato fuori dai rami**, dove il valore non cambia: con if (i > 0) ... else if (i < 0) il caso 0 non assegnava niente → 0/0 a test verdi (§1.3).
- ⚠ **Niente di ridondante**: un boolean si restituisce (return cond;, non if (cond) return true; else return false;); ciò che il costruttore garantisce (den > 0) non si ricontrolla.
- ⚠ Un metodo che risponde (equals, getter) **non stampa**.
- equals(Frazione f) è un metodo nuovo, non ridefinisce l>equals(Object) di ogni classe: la forma completa arriva con LAB09-LAB10.

mcm da mcd — MyMath dello startkit:

```

public static int mcm(int a, int b) {
    return (a * b) / mcd(a, b); // il prodotto contiene la parte comune due vol
}                               // (!) non "- mcd": mcm(4,6) darebbe 22. Prova:

```

mcd(a, b) (Euclide, già nello startkit) vuole **naturali, non 0** [LAB02 sl. 9]: passare Math.abs(...) e gestire lo 0 prima (§1.2). a * b può andare in overflow prima della divisione (§3.3).

3.9 Operazioni che restituiscono un oggetto nuovo (LAB03)

L'mcm, dalla base. Due frazioni si sommano solo se i pezzi hanno la **stessa grandezza**, cioè lo stesso denominatore. $mcm(a, b)$ è il più piccolo numero in cui stanno esattamente sia a sia b .

Passo	4 e 6	Perché
scomponi	$4 = 2 \cdot 2, 6 = 2 \cdot 3$	
moltiplica	$4 \cdot 6 = 24 = 2 \cdot 2 \cdot 2 \cdot 3$	il 2 comune compare due volte, una per numero
dividi per l'mcd	$24 / mcd(4,6) = 24 / 2 = 12 = 2 \cdot 2 \cdot 3$	l'mcd è la parte comune: la tieni una volta sola

Da qui $mcm(a, b) = a \cdot b / mcd(a, b)$ (§3.8). **Portare 1/4 in dodicesimi:** quante volte il 4 sta nel 12? $12 / 4 = 3 \rightarrow 1/4 = (1 \cdot 3)/(4 \cdot 3) = 3/12$; il valore non cambia perché moltiplichiamo sopra e sotto per lo stesso numero. Il fattore è mcm / den : il multiplo diviso il denominatore, mai il contrario ($4 / 12$ fra int fa 0). Prova: $1/4 + 1/6 \rightarrow mcm 12 \rightarrow 3/12 + 2/12 = 5/12$.

Nella forma del docente, verificata coi test di LAB03 [LAB03 sl. 4-7]. Il perché: esame_FI2/svolti/LAB03_Frazione/confronto_LAB03.md.

```
public Frazione sum(Frazione f) {
    int mcm = MyMath.mcm(f.getDen(), this.getDen());
    int n = ((mcm / this.getDen()) * this.getNum())
        + ((mcm / f.getDen()) * f.getNum());
    return (new Frazione(n, mcm)).minTerm();
}

public Frazione sub(Frazione f) {
    int mcm = MyMath.mcm(f.getDen(), den);
    int n = ((mcm / den) * num) - ((mcm / f.getDen()) * f.getNum());
    return new Frazione(n, mcm).minTerm();
}

public Frazione mul(Frazione f) {
    int n = this.getNum() * f.getNum();
    int d = this.getDen() * f.getDen();
    return new Frazione(n, d).minTerm();
}

public Frazione reciprocal() {
    return new Frazione(getDen(), getNum()).minTerm();
}

public Frazione div(Frazione f) {
    return mul(new Frazione(f.getDen(), f.getNum())).minTerm();
}
```

// this + f, SENZA modificare f
// denominatore comune (mcm)
// numeratore di this portato a mcm
// idem per f
// oggetto NUOVO, ridotto a mcm

// this - f: nell'ordine mcm
// dentro la classe si può usare den
// parte di f da sottrarre

// prodotto: numeratori e denominatori
// il costruttore sistema tutto

// scambia num e den
// -2/3 → Frazione(3, -2)
// (!) reciproco di 0/n = 0

// dividere = moltiplicare per il reciproco
// RIUSA mul: nessuno si toglie il denominatore

```

public int compareTo(Frazione f) {
    int thisValue = this.getNum() * f.getDen();
    int otherValue = f.getNum() * this.getDen();
    if (thisValue == otherValue) return 0;
    else return thisValue > otherValue ? 1 : -1;
}

public double getDouble() {
    return (double) getNum() / (double) getDen();
}

```

// 0 se uguali, 1 se this > f, -1 se this < f
// a/b ? c/d ⇔ a·d ? c·b
// ... valido SOLO perche
// stessa formula di equa
// interi: esatto, niente

// cast PRIMA della /: ba
// (!) (double)(num / den)

Espressione (int 3 e 12)	Valore	
3 / 12	0	divisione intera
(double) 3 / 12	0.25	il cast lega a 3, poi divisione reale
(double) (3 / 12)	0.0	il cast arriva dopo la divisione intera
3.0 / 12	0.25	un letterale double basta

- ⚠ **Il fattore è mcm / den** (verso: profilo/errori.md pattern 5). Controllo: $1/4 + 1/8$ deve dare $3/8$.
- ⚠ **Il test non basta:** FrazioneTest controlla compareTo solo sul caso 0 e sub su un caso dove due errori si compensano. Aggiungi $1/2$ contro $1/3$ ($\rightarrow 1$), il contrario ($\rightarrow -1$), $1/2 - 1/3$ ($\rightarrow 1/6$).
- sum alternativa, senza mcm (sl. 6): $n = \text{num} \cdot f.\text{den} + \text{den} \cdot f.\text{num}$, $d = \text{den} \cdot f.\text{den}$, poi minTerm(). All'esame si usa il metodo che chiede il testo.
- Dentro la classe f.den compila anche se den è private: private protegge **dalle altre classi**, non dagli altri oggetti della stessa classe [sl. 6].

3.10 Package, libreria static, array di oggetti (LAB04a)

Svolto: esame_FI2/svolti/LAB04a_FrazioniBase/src/. Fonti: LAB04 sl. 4-21, 08 sl. 18-20, 07 sl. 17-19.

static o d'istanza? Chiediti: *il metodo usa this?*

	Metodo d'istanza	Funzione static
si chiama su	un oggetto : a.sum(b)	la classe : FrazLib.sum(fs), MyMath.mcd(x, y)
this	c'è: è l'oggetto prima del punto	non esiste
esempio	Frazione.sum(Frazione f): this + f	FrazLib.sum(Frazione[] fs): lavora solo sui parametri

sum di un array non sta in Frazione (non parte da *una* frazione) né nell'array (gli array non hanno metodi tuoi): sta in un «ente terzo», la libreria statica [LAB04 sl. 5-9].

```
package frazlib;                                // prima riga: il package della classe (=
import frazione.Frazione;                       // Frazione sta in un altro package: senza
public class FrazLib {                          // libreria: solo metodi static, nessun c

    public static Frazione sum(Frazione[] fs) { // static: si chiama FrazLib.sum
        Frazione tmp = new Frazione(0, 1);     // accumulatore = ELEMENTO NEUTRO
        for (Frazione f : fs)                 // for each: f vale fs[0], fs[1], ...
            tmp = tmp.sum(f);                  // riusa la somma a due di Frazione
        return tmp;                           // array vuoto → 0/1; risultato g
    }

    public static Frazione mul(Frazione[] fs) { // (!) neutro del PRODOTTO è 1: p
        Frazione tmp = new Frazione(1, 1);     // (!) mul, non sum: rileggi ogni
        for (Frazione f : fs)                 // array vuoto → 1
            tmp = tmp.mul(f);
        return tmp;
    }
}
```

Array di oggetti	Cosa fa
Frazione[] fs = new Frazione[4];	4 caselle null : nessuna frazione è ancora stata creata [07 sl. 17]
fs[0] = new Frazione(1, 3);	riempie la casella 0
fs.length	numero di caselle — campo , senza () (le String invece hanno length())
for (int i = 0; i < fs.length; i++)	serve l'indice (posizione, celle vicine)
... fs[i] ...	
for (Frazione f : fs) ... f ...	basta il valore: più corto, niente errori di indice
Frazione[]	esiste da sé per ogni classe: non si dichiara da nessuna parte

Verificato (javac): sum e mul di un array vuoto → 0 e 1; sum({3/6}) → 1/2.

3.11 Array riempito a metà, metodi statici dentro la classe-tipo (LAB04b)

Svolto: esame_FI2/svolti/LAB04b_FrazioniDoubleFace/src/frazione/Frazione.java.
Fonti: LAB04 sl. 22-40, 07 sl. 14, 05 sl. 57.

Fine fisica e fine logica. new Frazione[10] con 4 frazioni: length = 10 (celle esistenti), dimensione **logica** = 4 (celle usate). Convenzione del corso: riempimento

in sequenza, il **primo null segna la fine** [LAB04 sl. 32]. $\triangle$ Tre tentativi sbagliati prima di quello giusto:

Condizione	Array pieno {1/2, 1/3}	Array a metà
<code>i < fs.length - 1</code>	salta l'ultima $\rightarrow$ 1/2	NullPointerException
<code>i < fs.length</code>	✓	NullPointerException
<code>i < fs.length && fs[i] != null</code>	✓	✓ — questa, sempre [LAB04 sl. 28]

L'**ordine** conta: con `&&`, se `i < fs.length` è falso la seconda non viene valutata, quindi `fs[length]` non viene mai letto. Invertite $\rightarrow$ `ArrayIndexOutOfBoundsException` su un array pieno.

Tre sum nella stessa classe (overloading: stesso nome, argomenti diversi [05 sl. 57]):

Chiamata	Prende	Restituisce
<code>f.sum(g)</code> — d'istanza	due frazioni	una Frazione
<code>Frazione.sum(tutte)</code> — static	un array	una Frazione, il totale
<code>Frazione.sum(setA, setB)</code> — static	due array	un array , cella per cella

```

public static int size(Frazione[] fs) {                                // dimensione LOGICA: celle
    int size = 0;
    for (size = 0; size < fs.length && fs[size] != null; size++) {    // corpo vuoto: il contatore
    }                                                                    // pieno  $\rightarrow$  length; tutto null
    return size;                                                        //
}

public static Frazione[] sum(Frazione[] fA, Frazione[] fB) {
    if (size(fA) != size(fB)) return null;                            // dimensioni logiche diverse
    Frazione[] risultato = new Frazione[size(fA)];                    // creato DOPO aver saputo
    for (int i = 0; i < risultato.length; i++)                         // qui length = dimensione
        risultato[i] = fA[i].sumWithMcm(fB[i]);                       // (!) via mcm: il test vuoto
    return risultato;                                                  // array NUOVO: fA e fB restano
}                                                                    // mul a coppie: identica,

public static String convertToString(Frazione[] fs) {
    String res = "[";
    for (int i = 0; i < fs.length && fs[i] != null; i++) {            // stessa condizione di
        if (i != 0) res += ", ";                                       // separatore PRIMA, tra
        res += fs[i].toString();                                        // fs[i] qui non è mai null
    }
    res += "]";                                                        // vuoto o tutto null -
    return res;
}

```

Verificato (javac): size → 4 / 2 / 0 su array del test / pieno / vuoto; sum a coppie del test → [8/15, 11/12, -5/14, 0/6]; dimensioni 4 e 2 → null; convertToString → [1/3, 2/3, -1/2, 1/6]. $\triangle$ Il sum(Frazione[]) della soluzione del docente usa il *for each* e va in NullPointerException sugli array a metà: per gli array a metà usa la condizione sopra.

3.12 ADT con array nascosto: campi, costruttori, put, remove, toString (LAB04c)

Una classe che nasconde un array (FractionCollection, sl. 59). Svolta il 2026-10-02, **guidato**; test verdi con -ea. Il confronto con il docente: esa-me_FI2/svolti/LAB04c_FractionCollection/confronto_LAB04c.md.

Riga UML (sl. 59)	Java	Perché
- DEFAULT_PHYSICAL_SIZE: int = 10 {readOnly} (sottolineata)	private static final int DEFAULT_PHYSICAL_SIZE = 10;	{readOnly} = final; sottolineato = static (una per classe). Si inizializza sulla riga
- innerContainer: Frazione[]	private Frazione[] innerContainer;	l'array di supporto; i campi normali non hanno = ... (partono a null / 0) e si inizializzano nel costruttore
- size: int	private int size;	dimensione logica = quante frazioni ci sono = indice della prima cella libera

La dimensione fisica **non è un campo**: è innerContainer.length. physicalSize esiste solo come parametro. Regola che tiene in piedi la classe: ogni punto che cambia il contenuto (tre costruttori, put, remove) tiene size coerente; size() restituisce solo il campo.

Costruttori

```

public FractionCollection(int physicalSize) {
    innerContainer = new Frazione[physicalSize]; // crea lo "scaffale": physicalSize
    size = 0; // logicamente vuota: nessuna frazione
}

public FractionCollection() {
    innerContainer = new Frazione[DEFAULT_PHYSICAL_SIZE]; // capacità di default,
} // (alternativa del docente)

public FractionCollection(Frazione[] collection) {
    size = Frazione.size(collection); // dimensione LOGICA del parametro
}

```

```

        innerContainer = new Frazione[size];
        for (int i = 0; i < size; i++) {
            innerContainer[i] = collection[i];
        }
    }

```

*// array nuovo, lungo quanto se
 // copia solo le celle occupate
 // `collection` = parametro, `in*

Accesso: size, get

```

public int size() { return size; }
// getter: il campo è private, c

public Frazione get(int index) {
    if (index < 0 || index >= size)
        throw new IndexOutOfBoundsException("indice " + index + " fuori da 0.." + c
    return innerContainer[index];
}
// ||: sbagliato se negativo 0P
// dopo il throw il metodo non p

```

△ `index < 0 && index >= size` è sempre falso: il throw non scatta mai. △ Non ricalcolare `size()` contando i null: con get corretto il ciclo chiamerebbe get oltre la fine logica.

put: aggiunge in coda, raddoppia se pieno

```

public void put(Frazione f) {
    if (innerContainer.length == 0)
        innerContainer = new Frazione[DEFAULT_PHYSICAL_SIZE];
    if (size == innerContainer.length) {
        Frazione[] fs = new Frazione[innerContainer.length * DEFAULT_GROWTH_FACTOR];
        for (int i = 0; i < size; i++)
            fs[i] = innerContainer[i];
        innerContainer = fs;
    }
    innerContainer[size] = f;
    size++;
}
// capacità 0.
// array pieno
// copia le fr
// (!) il nuov
// inserimento
// e `size` ag

```

Dopo un raddoppio restano celle null libere per le put successive: è la capacità di riserva, e per questo esiste `size`. Raddoppiare (e non crescere di 1) evita di ricopiare l'array a ogni put.

remove(index): sposta indietro, azzerà l'ultima, size--

```

public void remove(int index) {
    if (index < 0 || index >= size)
        throw new IndexOutOfBoundsException("indice " + index + " fuori da 0.." + c
    for (int i = index; i < size - 1; i++)
        innerContainer[i] = innerContainer[i + 1];
    innerContainer[size - 1] = null;
    size--;
}
// < size, non
// si ferma un
// la cella i
// la vecchia

```

Con [A, B, C] e remove(1) → [A, C, null], size 2. Se index è l'ultimo il ciclo non gira e si azzerava solo l'ultima cella. ⚠ Con `i < size` e array pieno: `ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3`; con `size - 1` azzerato a `[size]`, l'ultima frazione resta in doppio.

Operazioni fra due collezioni: sum, mul

```
public FractionCollection sum(FractionCollection other) {
    if (this.size != other.size) // «di pari dimensioni»
        throw new IllegalArgumentException("dimensioni diverse: " + this.size + " e " + other.size);
    FractionCollection res = new FractionCollection(this.size); // risultato
    for (int i = 0; i < this.size; i++) // fino a size
        res.put(this.get(i).sumWithMcm(other.get(i))); // put aggiornato
    return res;
}
// mul: identico, con .mul(...) al posto di .sumWithMcm(...)
```

Non sostituire `innerContainer` di `this`: il metodo **restituisce** una collezione nuova.

toString con StringBuilder

```
@Override
public String toString() {
    StringBuilder sb = new StringBuilder("["); // una String non si modifica
    for (int i = 0; i < size; i++) { // solo le celle piene
        if (i != 0) sb.append(", "); // la virgola va PRIMA di ogni elemento
        sb.append(innerContainer[i]); // append converte con toString
    }
    sb.append("]");
    return sb.toString(); // alla fine: la String
}
```

`size 0` → il ciclo non gira → `[]`; un elemento → `[1/2]`; `2/1` si stampa `2`. ⚠ Con il ciclo fino a `size - 1` e l'ultimo elemento fuori: `ArrayIndexOutOfBoundsException: Index -1 out of bounds for length 10` sulla collezione vuota.

Eccezioni lanciate qui

Si lancia	Quando	Sintassi
<code>IndexOutOfBoundsException</code>	indice fuori da <code>0 ... size-1</code> (get, remove)	<code>throw new IndexOutOfBoundsException("messaggio");</code>
<code>IllegalArgumentException</code>	argomento che non rispetta il contratto (sum/mul con size diverse)	<code>throw new IllegalArgumentException("messaggio");</code>

Sono non controllate (RuntimeException): niente throws nella firma né try/catch. Il docente, invece, restituisce null / esce in silenzio: scelta del testo non specificata, quella con l'eccezione non nasconde l'errore.

Verificato eseguendo (javac/java, JDK 21): FractionCollectionTests con -ea verde; $[1/2, 1/3] + [1/2, 1/6] = [1, 1/2]$; $\times = [1/4, 1/18]$; originali invariati; vuota + vuota $\rightarrow$ size 0; put su capacità 0 $\rightarrow$ size 1.

3.13 java.time, formattatori, regola a passi (LAB05)

Svolto: esame_FI2/svolti/LAB05_TicketSosta/src/ticketsosta/ (Ticket.java, Parcometro.java). Fonti: LAB05 sl. 11-13, 8.

Cosa va nel costruttore e cosa nei parametri. Il costruttore riceve ciò che **resta vero per tutta la vita dell'oggetto** (il Parcometro ha la sua Tariffa); il metodo riceve i dati **di una singola chiamata** (emettiTicket(inizio, fine)). Un Ticket nasce *dentro* emettiTicket, con i valori dei parametri; non servono nel costruttore del Parcometro.

```
public class Parcometro {
    private Tariffa tariffa; // stato: la tariffa della zona

    public Parcometro(Tariffa tariffa) { // arriva una volta, alla creazione
        this.tariffa = tariffa;
    }

    public Ticket emettiTicket(LocalTime inizio, LocalTime fine) { // arrivano a
        double costo = calcolaCosto(tariffa.getTariffaOraria(), inizio, fine); //
        return new Ticket(inizio, fine, costo); // 2) il ticket, ordine = queue
    }
}
```

Formattatori — frammenti da copiare [LAB05 sl. 11]; verificati eseguendo (JDK 21).

Devo	Codice	Esce
un importo in euro	NumberFormat.getCurrencyInstance(Locale.ITALY).format(3.5)	3,50€ (non breakable), non " ": nel test "3,50\u00A0€"
un orario corto	orario.format(DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT))	08:30 (con MDTU e Time secondi)
la durata fra due orari	Duration.between(inizio, fine)	oggetto Duration, non un numero
...in minuti totali	Duration.between(da, a).toMinutes()	450 per 7:30→15:00 (tipo long)
...solo la parte dei minuti	duration.toMinutesPart()	30 per 6h30: non è il totale
...ore intere	duration.toHours()	6

Import: java.text.NumberFormat, java.util.Locale, java.time.format.DateTimeFormatter, java.time.format.FormatStyle, java.time.Duration, java.time.LocalDateTime.

```
private String toStringDuration(Duration duration) {
    long ore = duration.toHours();           // ore intere: 3 per 3h05
    long minuti = duration.toMinutesPart();  // solo i minuti restanti: 5
    if (minuti < 10) {
        return ore + ":0" + minuti;         // 3:05 – lo zero davanti
    } else return ore + ":" + minuti;       // 3:45
}
```

Regola «franchigia, poi minimo» [LAB05 sl. 8]. Δ L'ordine conta, e va confrontata la variabile **dopo** la sottrazione:

```
long minutiSosta = Duration.between(da, a).toMinutes(); // 1) durata totale, i
long minutiDaPagare = minutiSosta - tariffa.getMinutiFranchigia(); // 2) toglie i
if (minutiDaPagare < tariffa.getDurataMinima()) {           // 3) sotto il minimo
    minutiDaPagare = tariffa.getDurataMinima();             //   paghi il minimo
}
return costoOrario * (minutiDaPagare / 60.0);              // 4) ore con `60.0`
```

Verificato: H1 (0,50 €/h, franchigia 0, minimo 60) 7:30→15:00 = 3,75 €; H1f (franchigia 60) 7:30→15:00 = 3,25 €; H1f 10:00→11:30 = 0,50 €. **Fuori da questa versione:** sosta che finisce alle 00:00 o a cavallo di mezzanotte (la soluzione del docente la tratta con `a.isBefore(da) || LocalDateTime.of(0, 0).equals(a)`) → confronto in `svolti/LAB05_TicketSosta/confronto_LAB05.md`. Δ `LocalTime.plusMinutes` oltre le 24:00 **riparte da 00:00** senza avvisare.

Parte 2 — `LocalDateTime`, giorni della settimana, più giorni (LAB05, `ticketso-staevoluto`)

Svolto: `esame_FI2/svolti/LAB05_TicketSosta/src/ticketsostaevoluto/`. Fonti: LAB05 sl. 15–27. Le classi nuove stanno in un **package nuovo**: `Ticket` e `Parco-`
`metro` della parte 1 **non si modificano** (i loro test passano `LocalTime`).

Devo	Codice	Nota
il giorno della settimana di una data	<code>data.getDayOfWeek()</code> (su <code>LocalDate</code> e su <code>LocalDateTime</code>)	restituisce un <code>DayOfWeek</code> (MONDAY...), non un numero
...come indice di array (lunedì = 0 ... domenica = 6)	<code>data.getDayOfWeek().getValue()</code> va da 1 - 1 oppure <code>data.getDayOfWeek().ordinal()</code> da -	<code>getValue()</code> va da 1 (lunedì) a 7: derivato (seguendo, non a memoria. <code>ordinal()</code> dà già 0–6 (soluzione del docente)
la tariffa del giorno	<code>tariffa[giorno.getDayOfWeek().getValue() - 1]</code>	leggi il giorno che stai pagando , non da quello di partenza

Devo	Codice	Nota
solo la data / solo l'ora di un LocalDateTime due date nello stesso giorno?	dt.toLocalDate() / dt.toLocalTime() a.toLocalDate().equals(b.toLocalDate())	
aggiungere minuti / giorni	dt.plusMinutes(n) / data.plusDays(1)	△ restituisce un nuovo oggetto: da.plusDays(1); da solo non fa niente, va assegnato (giorno = giorno.plusDays(1);)
una Tariffa come testo, per ogni elemento	tariffa[i].toString()	△ tariffa.toString() su un array stampa [Lticketso- sta.Tariffa;@6d06d69c
un ParcometroEvoluto come testo	String s = "...\\n"; for (int i = 0; i < tariffa.length; i++) { s = s + tariffa[i].toString() + "\\n"; } return s;	length, non 7 scritto a mano; restituire , non stampare

Giorni intermedi interi: calcolaCosto(costoOrario, LocalTime.MIDNIGHT, LocalTime.MIDNIGHT) dà 24 ore, grazie al ramo LocalTime.of(0, 0).equals(a) del metodo del docente (1440 minuti). Verificato.

```
// sosta su più giorni: primo pezzo (con franchigia), giorni interi, ultimo pezzo
LocalDate giorno = da.plusDays(1).toLocalDate(); // il giorno
LocalDateTime daEffettivo = da.plusMinutes(tariffa[da.getDayOfWeek().getValue() - 1].getDurataMinima());
double res;
res = calcolaCosto(tariffa[da.getDayOfWeek().getValue() - 1].getTariffaOraria(),
                  daEffettivo.toLocalTime(), LocalTime.MIDNIGHT); // primo giorno
while (giorno.isBefore(a.toLocalDate())) { // giorni interi
    res = res + calcolaCosto(tariffa[giorno.getDayOfWeek().getValue() - 1].getTariffaOraria(),
                            LocalTime.MIDNIGHT, LocalTime.MIDNIGHT); // 24 ore
    giorno = giorno.plusDays(1); // avanza il giorno
}
return res + calcolaCosto(tariffa[a.getDayOfWeek().getValue() - 1].getTariffaOraria(),
                          LocalTime.MIDNIGHT, a.toLocalTime()); // ultimo pezzo
```

Franchigia e minimo su un solo giorno (stessa regola della parte 1, con la tariffa del giorno):

```
Tariffa t = tariffa[da.getDayOfWeek().getValue() - 1]; // tariffa
LocalDateTime daEffettivo = da.plusMinutes(t.getMinutiFranchigia()); // il conto
long minutiDaPagare = Duration.between(daEffettivo, a).toMinutes(); // negativo
if (minutiDaPagare < t.getDurataMinima()) { // sotto il minimo
    return t.getTariffaOraria() * t.getDurataMinima() / 60.0; // `60.0`,
```

```
}
return calcolaCosto(t.getTariffaOraria(), daEffettivo.toLocalTime(), a.toLocalTime
```

Verificato su tutti gli 8 casi di ParcometroEvolutoTest (incl. mer 11:30→11:50 = 1,50 €, ven→sab = 60,25 €, ven→gio 1/4 = 243,25 €). Δ Il **docente** decide il minimo **una volta, sull'intera sosta**, in emettiTicket (confronto, punto 10): mio, solo nel ramo «stesso giorno».

Δ Ticket, non compila dopo un quick fix: se scrivendo ParcometroEvoluto Eclipse propone «Change type of...» e accetti, può cambiare **la classe della parte 1** (Ticket: LocalTime → LocalDateTime) e Parcometro smette di compilare (LocalTime cannot be converted to LocalDateTime). Dopo ogni quick fix guarda quale file si è modificato.

4 — Persistence

Vuota: si riempie coi LAB che leggono e scrivono file e con le prove.

5 — Controller e UI (JavaFX)

Vuota: si riempie da 31x in poi e con le prove.

Appendice — Per l'orale (solo se lo chiedi)

	Java	C#	Scala / Kotlin
tipi base	primitivi , non oggetti	classi trattate come primitivi (int = Int32)	veri tipi-oggetto
chiedere un servizio	funzione statica di libreria: Charac-ter.toUpperCase(c)	anche metodo: ch.ToString(), 'B'.ToString()	metodo: x.toFloat (Scala senza parentesi), x.toFloat() (Kotlin)
stringa → numero	Integer.parseInt	Convert.ToInt32	toInt; senza eccezione toDoubleOption / toDoubleOrNull
main	in una classe, public static	in una classe	Scala in un object; Kotlin a livello di file

[02x sl. 2, 27, 31-35]. ⚠ Integer, Character di Java «NON VANNO CONFUSE COI TIPI-OGGETTO DI Scala e Kotlin» [02x sl. 27]: sono librerie **accanto** al primitivo, non al suo posto.